

Importing Libraries

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import mysql.connector
from scipy.stats import ttest_ind
import scipy.stats as stats
warnings.filterwarnings('ignore')
```

Loading the Dataset

In [3]:

```
db=mysql.connector.connect( host="localhost",
                           username="root",
                           password="PASSWORD@629",
                           database="inventory")

cur = db.cursor()

query= """select * from inventory.vendor_sales_summary;"""
cur.execute(query)
data=cur.fetchall()
data
columns = [desc[0] for desc in cur.description]

df=pd.DataFrame(data,columns=columns)
df
```

Out[3]:

	VendorNumber	VendorName	Brand	ActualPricePerProduct	PurPricePerProduct	TotalQuantityP
0	12546	JIM BEAM BRANDS COMPANY	1004	16.49	10.65	
1	12546	JIM BEAM BRANDS COMPANY	1004	16.49	10.65	
2	12546	JIM BEAM BRANDS COMPANY	1005	34.99	27.34	
3	2000	SOUTHERN WINE & SPIRITS NE	10058	13.99	9.26	
4	2000	SOUTHERN WINE & SPIRITS NE	10058	13.99	9.26	
...	...	...	...	...	...	...

	VendorNumber	VendorName	Brand	ActualPricePerProduct	PurPricePerProduct	TotalQuantityP
22281	2000	SOUTHERN GLAZERS W&S OF NE	27969	36.99	24.34	
22282	2000	SOUTHERN GLAZERS W&S OF NE	27969	36.99	24.34	
22283	10754	PERFECTA WINES	20439	13.99	4.68	
22284	12331	STELLAR IMPORTING CO LLC	263	17.99	13.95	
22285	3924	HEAVEN HILL DISTILLERIES	9123	0.99	0.74	

22286 rows × 17 columns

Exploratory Data Analysis

- Previously we examined the various tables of the database to identify key variables, understanding their relationship , and determine which ones should be included in the final table
- In this phase of EDA, we will analyze the resultant table to gain insights into the distribution of each column. This will help us understand data patterns, identify anomalies, and ensure data quality before further analysis

In [4]:

```
# Summary statistics:
df.describe().T
```

Out[4]:

	count	mean	std	min	25%	75%	max
VendorNumber	22286.0	9136.379925	16173.787011	2.000000e+00	3664.000000	4425.000000	2000.0
Brand	22286.0	18622.452975	12926.665223	5.800000e+01	6385.000000	18742.000000	27969
ActualPricePerProduct	22286.0	29.678403	94.274496	4.900000e-01	10.990000	15.990000	36.99
PurPricePerProduct	22286.0	19.972679	68.377515	3.600000e-01	6.997500	10.450000	24.34
TotalQuantityPurchased	22286.0	4493.971597	12609.807803	1.000000e+00	86.000000	804.000000	24.34
TotalAmountSpent	22286.0	48558.033029	173538.625094	7.100000e-01	1351.105000	9711.900000	1395.99
Volume	22286.0	858.172440	565.204188	5.000000e+01	750.000000	750.000000	24.34
SalesPricePerQuantity	22286.0	29.559151	94.173464	0.000000e+00	10.990000	15.990000	36.99
TotalQuantitySold	22286.0	1617.599794	6511.646728	1.000000e+00	37.000000	222.000000	24.34
TotalEarned	22286.0	22443.317608	79864.912647	0.000000e+00	875.422500	4377.285000	1395.99
TotalExcise	22286.0	888.249447	5374.757127	5.000000e-02	5.610000	39.130000	24.34
TotalFreight	22286.0	54088.315745	60873.719483	9.000000e-02	7982.650000	28720.520000	1395.99
GrossProfit	22286.0	-26114.715421	124197.884070	-3.509196e+06	-16055.782500	-1517.010000	1395.99

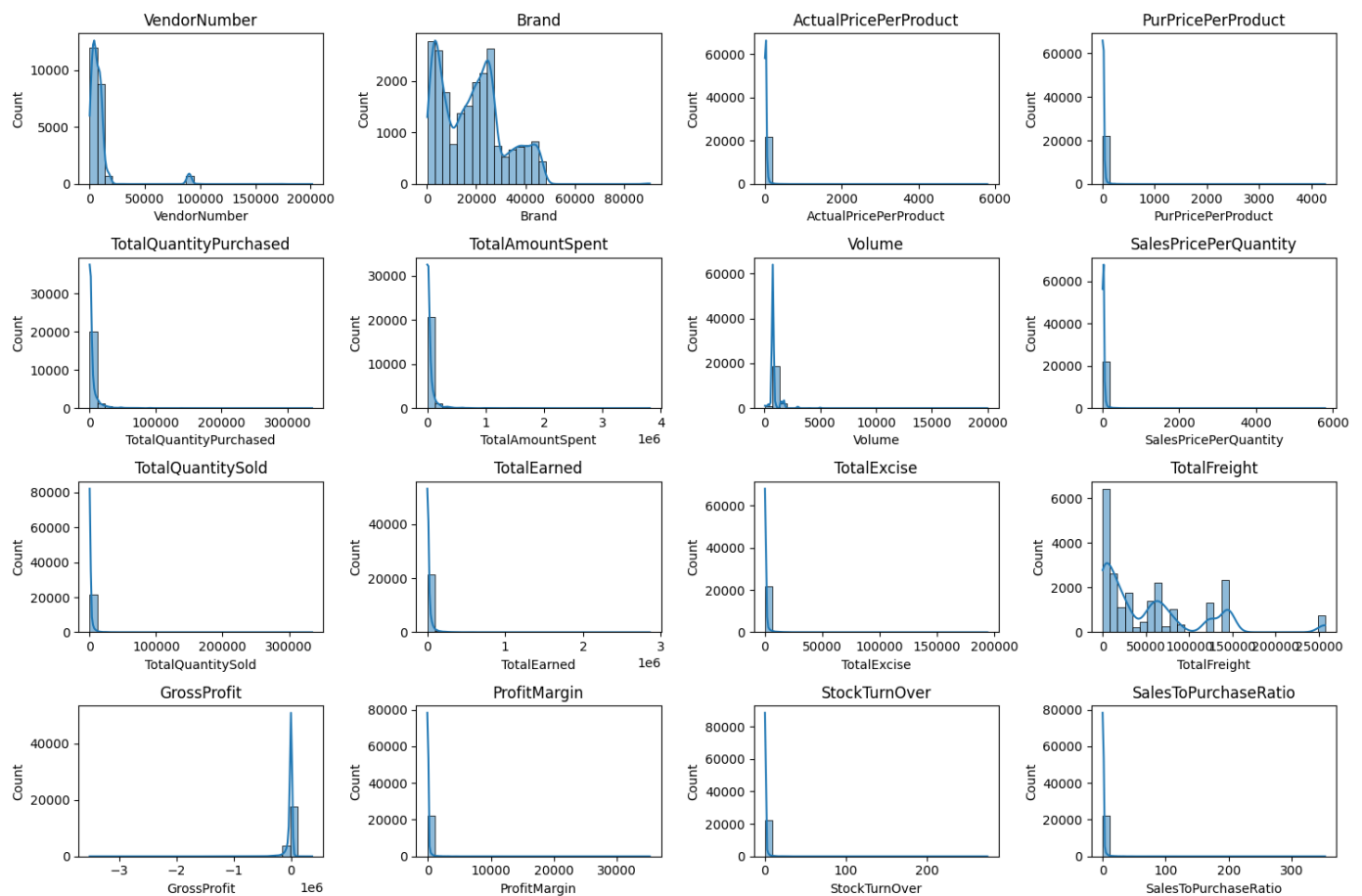
	count	mean	std	min	25%	75%
ProfitMargin	22286.0	44.213506	637.673264	-1.000000e+02	-75.464693	-36.481
StockTurnOver	22286.0	0.978747	4.503522	1.000000e-04	0.157025	0.423
SalesToPurchaseRatio	22286.0	1.442135	6.376733	0.000000e+00	0.245353	0.635

In [5]:

```
# Distribution of numerical columns:
```

```
numerical_cols=df.select_dtypes(include=np.number).columns
plt.figure(figsize=(15,10))
for i,col in enumerate(numerical_cols):
    plt.subplot(4,4,i+1)
    sns.histplot(df[col],kde=True,bins=30)
    plt.title(col)
```

```
plt.tight_layout()
plt.show()
```

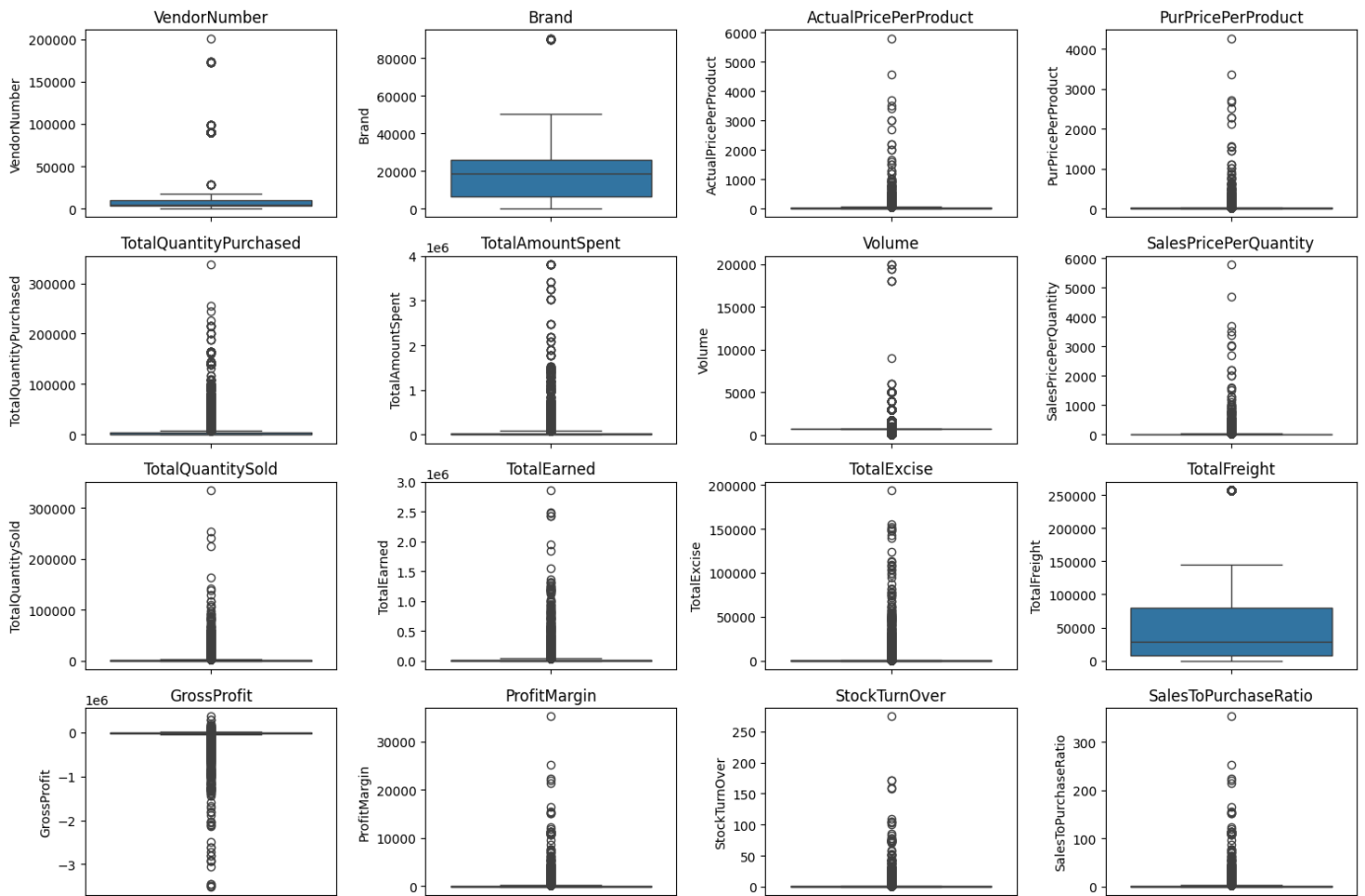


In [7]:

```
# Distribution of numerical columns:
```

```
numerical_cols=df.select_dtypes(include=np.number).columns
plt.figure(figsize=(15,10))
for i,col in enumerate(numerical_cols):
    plt.subplot(4,4,i+1)
    sns.boxplot(df[col])
    plt.title(col)
```

```
plt.tight_layout()
plt.show()
```



Summary Statistics Insights:

Negative And Zero Values:

- Gross Profit minimum value is -3.5×10^6 , Indicating losses or transactions may be selling at loss due to high costs or selling at discounts lower than the purchase price
- Total sales quantity and sales dollars are 0 for some products, meaning some products but never sold. These could be slow moving or obsolete stocks.

Outliers indicated by high standard deviations:

- Purchase price and actual price: The max values are significantly higher than the mean. Thus, indicating potential premium products.
- Freight costs: Huge variation from 0.09 to 257032. Suggests logistics inefficiency or bulk shipments.
- Stock Turnover: Ranges from 0.0001 to 274.5, implying some products sell very fast while some remain in stock indefinitely. Value more than 1 indicates that the sold quantity for that product is greater than the quantity purchased due to either sales are being fulfilled from older stocks.

In [8]:

```
# Let's filter out the data by removing the inconsistencies:
import mysql.connector
import pandas as pd
db=mysql.connector.connect( host="localhost",
```

```

        username="root",
        password="PASSWORD@629",
        database="inventory")

cur = db.cursor()

query= """SELECT * FROM vendor_sales_summary
WHERE GrossProfit>0

AND ProfitMargin>0

AND TotalQuantitySold>0;"""
cur.execute(query)
columns = [desc[0] for desc in cur.description]
data=cur.fetchall()
df=pd.DataFrame(data,columns=columns)
df

```

Out[8]:

	VendorNumber	VendorName	Brand	ActualPricePerProduct	PurPricePerProduct	TotalQuantityPu
0	12546	JIM BEAM BRANDS COMPANY	1005	34.99	27.34	
1	12546	JIM BEAM BRANDS COMPANY	1006	3.99	3.14	
2	10754	PERFECTA WINES	10062	8.99	6.03	
3	8352	LUXCO INC	1009	49.99	38.75	
4	17035	PERNOD RICARD USA	1013	13.99	10.29	
...	...	...	...	...	...	
7715	7153	PINE STATE TRADING CO	14761	36.99	24.02	
7716	1590	DIAGEO CHATEAU ESTATE WINES	22046	11.99	7.99	
7717	9165	ULTRA BEVERAGE COMPANY LLP	5485	29.99	21.42	
7718	90047	CRUSH WINES	15775	14.49	9.35	
7719	3924	HEAVEN HILL DISTILLERIES	9123	0.99	0.74	

7720 rows × 17 columns

In [9]:

```

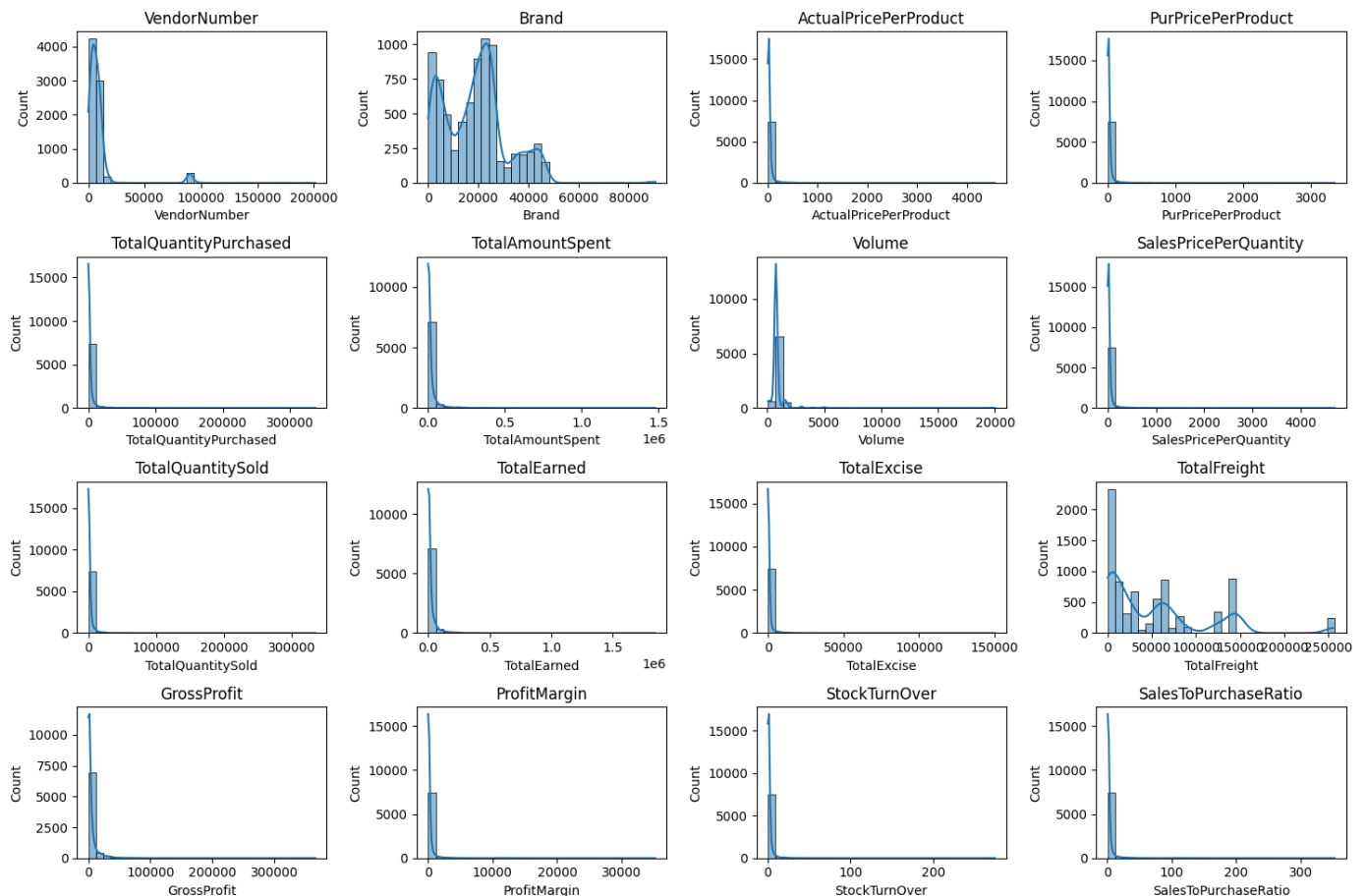
# Distribution of numerical columns after cleaning data:

numerical_cols=df.select_dtypes(include=np.number).columns

```

```
plt.figure(figsize=(15,10))
for i,col in enumerate(numerical_cols):
    plt.subplot(4,4,i+1)
    sns.histplot(df[col],kde=True,bins=30)
    plt.title(col)

plt.tight_layout()
plt.show()
```



In [10]:

```
df.describe().T
```

Out[10]:

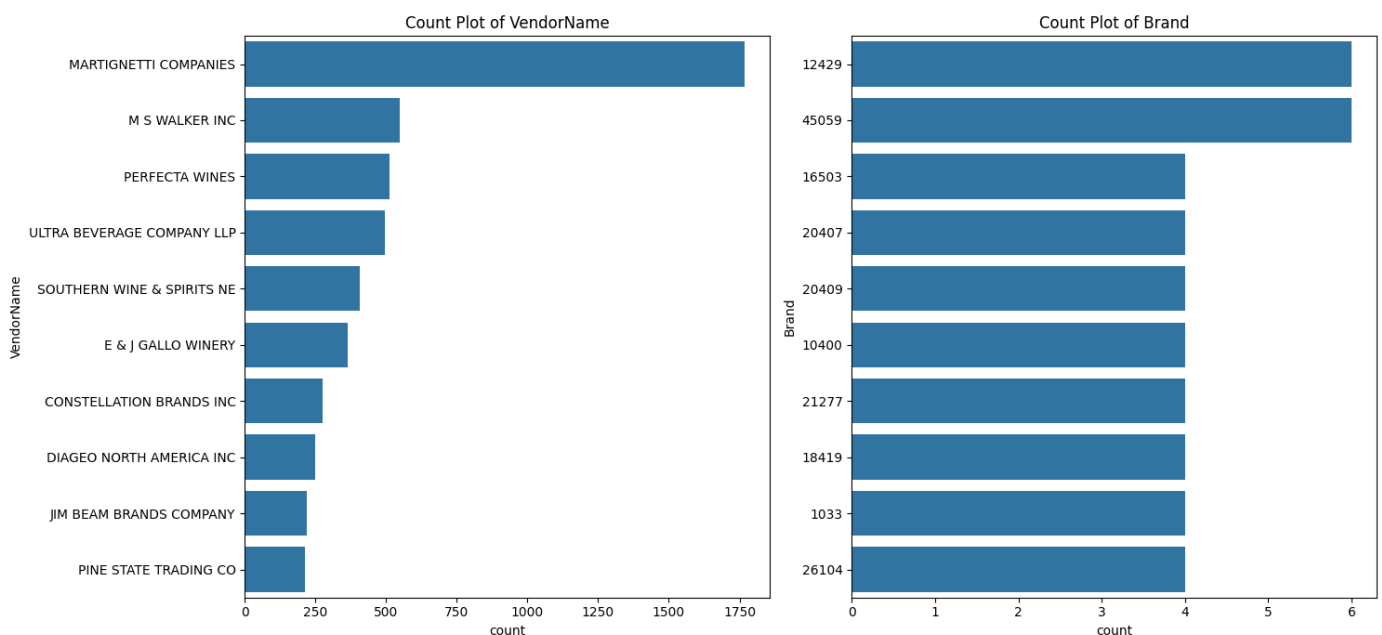
	count	mean	std	min	25%	50%
VendorNumber	7720.0	9475.159326	17141.877738	2.000000e+00	3950.000000	4425.000000
Brand	7720.0	18947.059326	12597.413805	5.800000e+01	7770.000000	19635.500000
ActualPricePerProduct	7720.0	36.363990	118.926430	4.900000e-01	9.990000	15.990000
PurPricePerProduct	7720.0	24.343764	84.678111	3.600000e-01	6.660000	10.270000
TotalQuantityPurchased	7720.0	2321.891321	9947.687041	1.000000e+00	24.000000	142.000000
TotalAmountSpent	7720.0	16289.566034	55536.932167	7.100000e-01	335.880000	1880.880000
Volume	7720.0	838.411075	754.075319	5.000000e+01	750.000000	750.000000
SalesPricePerQuantity	7720.0	35.789852	118.003402	4.900000e-01	9.990000	14.990000
TotalQuantitySold	7720.0	2114.456218	9492.748396	1.000000e+00	36.000000	173.500000
TotalEarned	7720.0	20979.137712	64971.700505	1.980000e+00	867.272500	3795.250000

	count	mean	std	min	25%	50%
TotalExcise	7720.0	807.643041	4899.385190	1.000000e-01	5.327500	31.340000
TotalFreight	7720.0	53018.183549	60546.547856	9.000000e-02	6015.860000	28720.520000
GrossProfit	7720.0	4689.571679	12069.610843	2.842171e-14	268.652500	1256.930000
ProfitMargin	7720.0	241.013824	1055.083389	1.367480e-14	24.750361	45.033796
StockTurnOver	7720.0	2.320786	7.464761	2.609000e-01	0.833300	0.993300
SalesToPurchaseRatio	7720.0	3.410138	10.550834	1.000000e+00	1.247504	1.450338

In [11]:

```
# Categorical Columns count plot:
categorical_cols = ["VendorName", "Brand"]
plt.figure(figsize=(15,7))
for i, col in enumerate(categorical_cols):

    plt.subplot(1,2,i+1)
    sns.countplot(y=df[col], order=df[col].value_counts().index[:10]) #Top 10 categories
    plt.title(f"Count Plot of {col}")
plt.tight_layout()
plt.show()
```

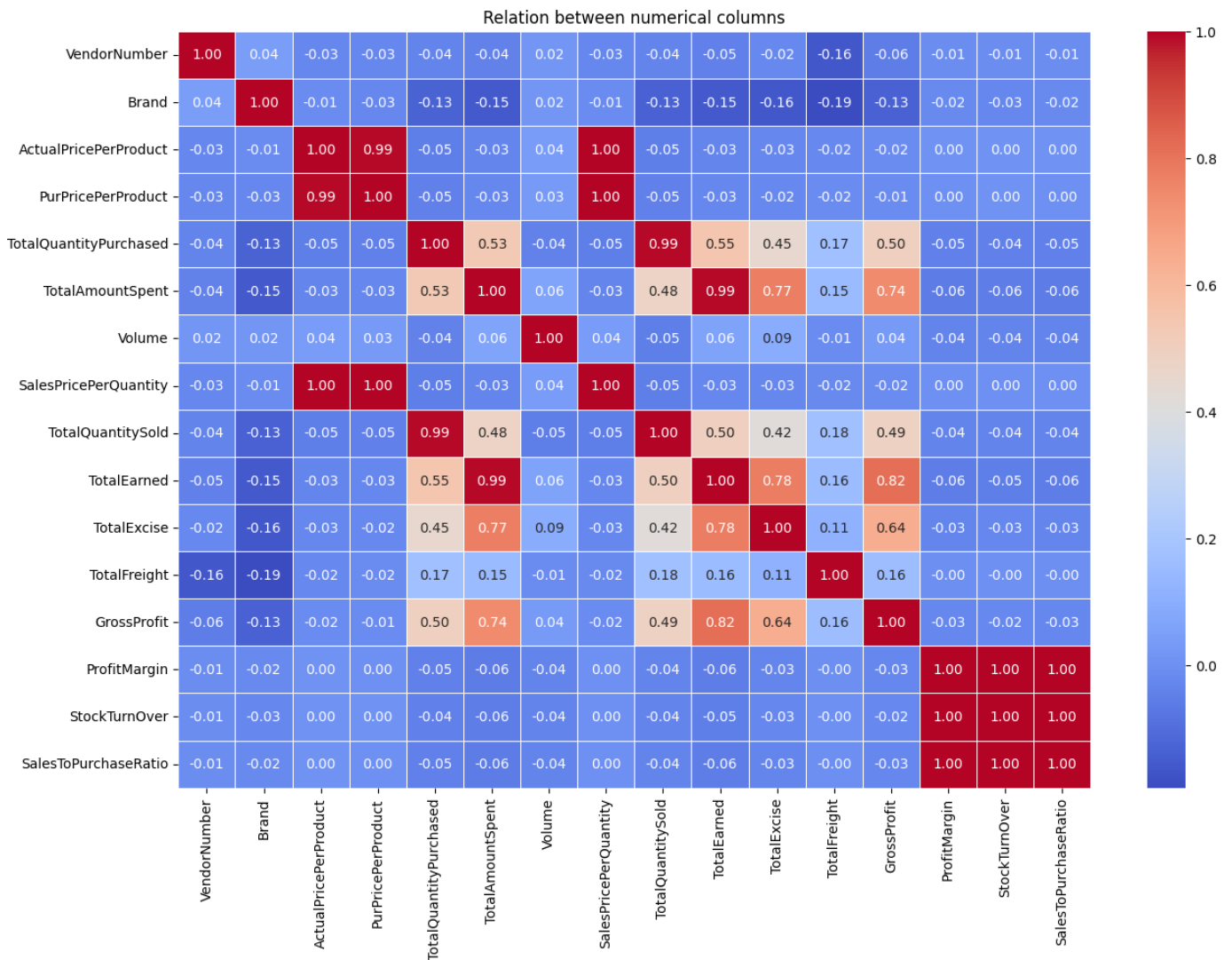


Relation between the numerical columns:

In [12]:

```
# Correlation Heatmap is the best way to show the relation between two numeric columns:

plt.figure(figsize=(15,10))
correlation_matrix = df[numerical_cols].corr()
sns.heatmap(correlation_matrix,annot=True,fmt='.2f',cmap='coolwarm',linewidths=0.5)
plt.title("Relation between numerical columns")
plt.show()
```



Correlation Insights:

- PurchasePrice has weak correlation with TotalEarned and GrossProfit, Suggesting that price variations donot significantly impact sales revenue or profit
- Strong correlation between TotalQuantityPurchased and TotalQuantitySold confirming efficient inventory turnover
- Negative correlation between profit margin & total sales price suggests that as sales price increases, margins decreases , possibly due to competitive pricing pressures
- StockTurnover has negative weak correlation with GrossProfit and ProfitMargin, indicating that faster turnovers does not necessarily result in higher profitability.

Identify the brands that need promotional or pricing adjustments which exhibit lower sales performance but higher profit margins

In [13]:

```
brand_performance = df.groupby('Brand').agg({'TotalEarned' : 'sum',
                                             'ProfitMargin' : 'mean'}).reset_index()
```



```
low_sales_threshold= brand_performance['TotalEarned'].quantile(0.15)
# Value of Total Earned below which 15% of the brands lie
high_profitMargin_threshold= brand_performance['ProfitMargin'].quantile(0.85)
#Value of ProfitMargin below which 85% of the total brands lie
brand_performance
```

Out[13]:

	Brand	TotalEarned	ProfitMargin
0	58	35527.65	7.842551
1	61	4364.88	31.981132
2	70	349.86	79.011461
3	75	314.79	187.742230
4	82	24749.34	453.661647
...	...	...	...
6391	90088	3509.74	532.659168
6392	90089	19078.41	665.144138
6393	90090	9749.85	262.499163
6394	90609	1474.41	8573.000000
6395	90631	31504.41	27.599258

6396 rows × 3 columns

In [12]:

```
low_sales_threshold
```

Out[12]:

```
np.float64(431.64)
```

In [13]:

```
high_profitMargin_threshold
```

Out[13]:

```
np.float64(230.4689963481889)
```

In [14]:

```
target_brands=brand_performance[
    (brand_performance['TotalEarned']<=low_sales_threshold) &
    (brand_performance['ProfitMargin']>=high_profitMargin_threshold)
]
print("Brands with low sales but high profit margin")
display(target_brands.sort_values("TotalEarned"))
```

Brands with low sales but high profit margin

	Brand	TotalEarned	ProfitMargin
704	2626	27.86	880.985915
1559	6064	27.96	460.320641
1495	5654	35.97	277.836134
3905	21691	44.94	769.245648

	Brand	TotalEarned	ProfitMargin
1400	5224	47.45	570.197740
...	...	...	...
1857	8151	410.85	7133.274648
540	2027	411.82	311.984794
5995	41417	415.74	233.766859
2984	17911	419.30	497.208375
3593	20466	419.93	242.995998

112 rows × 3 columns

In [15]:

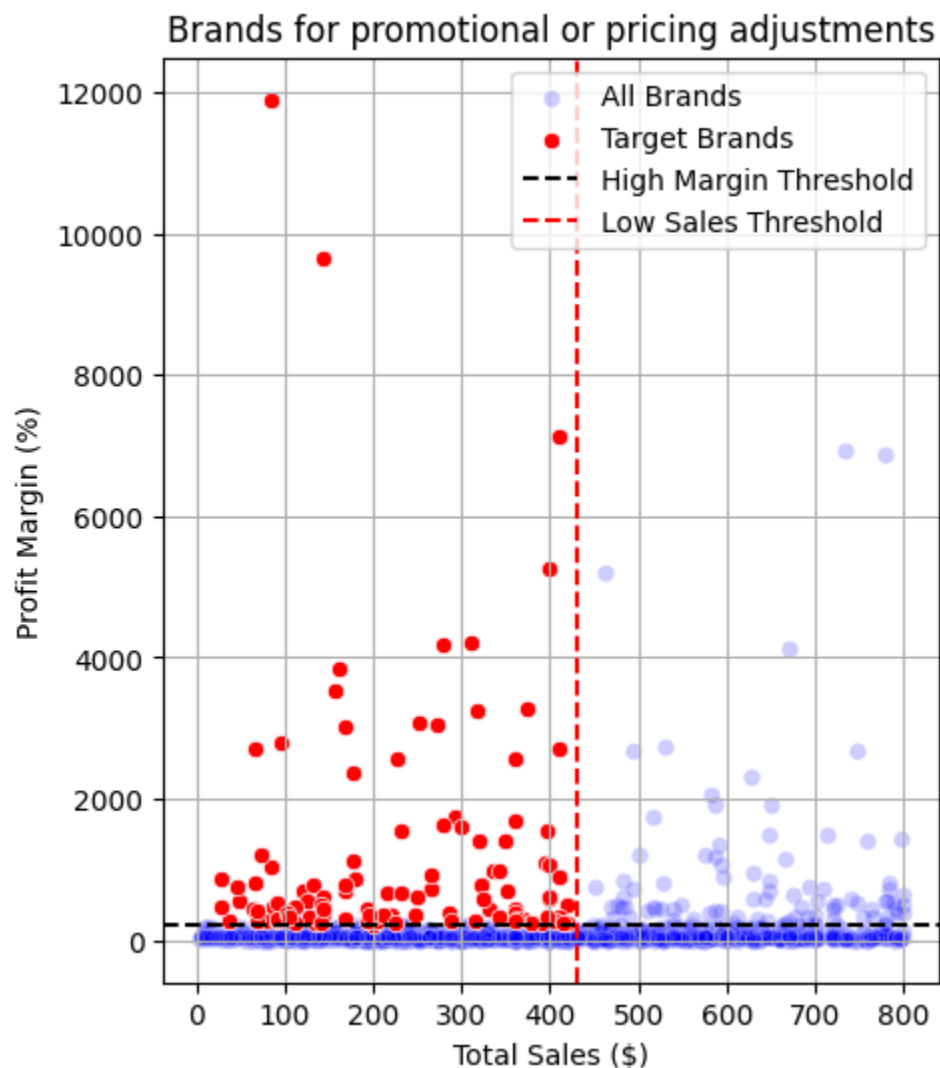
```
brand_performance=brand_performance[brand_performance["TotalEarned"]<800]
```

Scatter Plot Insight

- It can be seen that the brands which have earned less than around 400\$ to the vendor and which have a profit margin greater than around 240, need promotional and pricing adjustments

In [16]:

```
plt.figure(figsize=(5,6))
sns.scatterplot(data=brand_performance,x='TotalEarned',y='ProfitMargin',color='blue',label='T
sns.scatterplot(data=target_brands,x='TotalEarned',y='ProfitMargin',color='red',label='T
plt.axhline(high_profitMargin_threshold,linestyle='--',color='black',label='High Margin
plt.axvline(low_sales_threshold,linestyle='--',color='red',label='Low Sales Threshold')
plt.xlabel("Total Sales ($)")
plt.ylabel("Profit Margin (%)")
plt.title("Brands for promotional or pricing adjustments")
plt.legend()
plt.grid(True)
plt.show()
```



Vendors and Brands With highest sales performance

In [17]:

```
def format_dollars(value):
    if value >=1_000_000:
        return f"{value/1_000_000:.2f}M"
    elif value >=1_000:
        return f"{value/1_000:.2f}K"
    else:
        return str(value)
```

In [18]:

```
top_vendors=df.groupby("VendorName")["TotalEarned"].sum().reset_index().nlargest(10, "TotalEarned")
top_brands=df.groupby("Brand")["TotalEarned"].sum().reset_index().nlargest(10, "TotalEarned")
display(top_vendors)
display(top_brands)
```

	VendorName	TotalEarned
57	MARTIGNETTI COMPANIES	26914416.26
25	DIAGEO NORTH AMERICA INC	19936854.15

	VendorName	TotalEarned
46	JIM BEAM BRANDS COMPANY	9371071.64
30	E & J GALLO WINERY	9069167.64
20	CONSTELLATION BRANDS INC	8278742.67
68	PERNOD RICARD USA	8147211.91
6	BACARDI USA INC	7079081.85
106	ULTRA BEVERAGE COMPANY LLP	6358237.23
53	M S WALKER INC	6335293.11
79	SAZERAC CO INC	6315314.75

	Brand	TotalEarned
980	3505	2498514.30
714	2663	1843815.19
847	3040	1416801.42
1109	3857	1299590.03
1929	8512	1193542.93
1102	3826	1126056.69
398	1232	848422.96
1119	3893	816412.77
579	2127	797712.05
988	3522	773372.64

In [20]:

```
top_brands["TotalEarned"].apply(lambda x: format_dollars(x))
```

Out[20]:

```
980      2.50M
714      1.84M
847      1.42M
1109     1.30M
1929     1.19M
1102     1.13M
398      848.42K
1119     816.41K
579      797.71K
988      773.37K
```

Name: TotalEarned, dtype: object

In [21]:

```
plt.figure(figsize=(15,5))

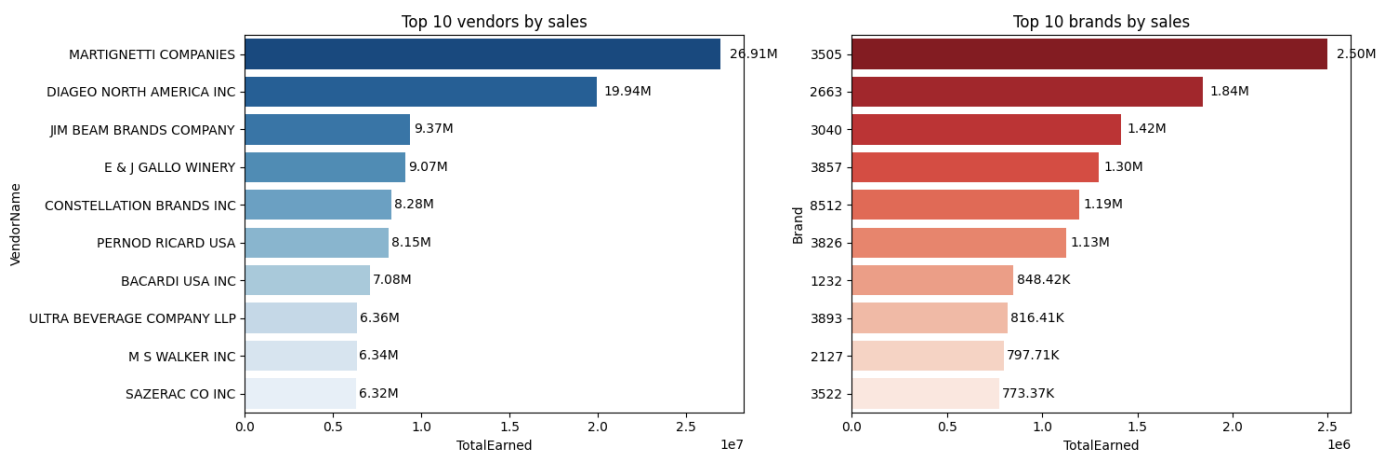
#plot for top vendors
plt.subplot(1,2,1)
ax1 = sns.barplot(y=top_vendors["VendorName"],
                  x=top_vendors["TotalEarned"],palette="Blues_r")
plt.title("Top 10 vendors by sales")
```

```

for bar in ax1.patches:
    ax1.text(bar.get_width()+(bar.get_width()*0.02),
             bar.get_y() + bar.get_height()/2,
             format_dollars(bar.get_width()),
             ha='left',va='center',fontsize=10,color='black')

#plot for top brands
plt.subplot(1,2,2)
ax2 = sns.barplot(y=top_brands["Brand"].astype(str),x=top_brands["TotalEarned"],palette=
plt.title("Top 10 brands by sales")
for bar in ax2.patches:
    ax2.text(bar.get_width()+(bar.get_width()*0.02),
             bar.get_y() + bar.get_height()/2,
             format_dollars(bar.get_width()),
             ha='left',va='center',fontsize=10,color='black')
plt.tight_layout()
plt.show()

```



Vendors Contributing The Most To total Purchases

In [22]:

```

Vendor_performance=df.groupby("VendorName").agg({
    'TotalEarned': 'sum',
    'GrossProfit': 'sum',
    'TotalAmountSpent': 'sum',
}).reset_index()

```

Vendor_performance

Out[22]:

	VendorName	TotalEarned	GrossProfit	TotalAmountSpent
0	ADAMBIA IMPORTS INTL INC	704.53	258.37	446.16
1	ALISA CARR BEVERAGES	104470.94	78772.82	25698.12
2	ALTAMAR BRANDS LLC	15706.81	4000.61	11706.20
3	AMERICAN SPIRITS EXCHANGE	1511.16	577.08	934.08
4	AMERICAN VINTAGE BEVERAGE	139603.53	35167.85	104435.68
...	...	...	...	...
114	WEIN BAUER INC	6130.65	2088.49	4042.16

	VendorName	TotalEarned	GrossProfit	TotalAmountSpent
115	WESTERN SPIRITS BEVERAGE CO	73929.40	18217.73	55711.67
116	WILLIAM GRANT & SONS INC	1608665.96	214716.46	1393949.50
117	WINE GROUP INC	2863491.13	446377.61	2417113.52
118	ZORVINO VINEYARDS	124189.59	38066.88	86122.71

119 rows × 4 columns

In [26]:

```
Vendor_performance['PurchaseContribution%']=(Vendor_performance['TotalAmountSpent']/Vendor_performance['TotalEarned']).round(2)
Vendor_performance=round(Vendor_performance.sort_values('PurchaseContribution%',ascending=False),2)
Vendor_performance
```

Out[26]:

	VendorName	TotalEarned	GrossProfit	TotalAmountSpent	PurchaseContribution%
57	MARTIGNETTI COMPANIES	26914416.26	7360527.44	19553888.82	15.55
25	DIAGEO NORTH AMERICA INC	19936854.15	4038795.35	15898058.80	12.64
46	JIM BEAM BRANDS COMPANY	9371071.64	1513275.43	7857796.21	6.25
68	PERNOD RICARD USA	8147211.91	1338496.63	6808715.28	5.41
30	E & J GALLO WINERY	9069167.64	2305311.58	6763856.06	5.38
...	...	...	...	...	...
56	MARSALLE COMPANY	2500.61	2421.83	78.78	0.00
85	SILVER MOUNTAIN CIDERS	342.51	265.33	77.18	0.00
16	CAPSTONE INTERNATIONAL	246.87	192.23	54.64	0.00
27	DJINN SPIRITS LLC	36.99	8.65	28.34	0.00
35	FLAVOR ESSENCE INC	1474.41	1457.41	17.00	0.00

119 rows × 5 columns

In [27]:

```
most_purchasing_vendors=Vendor_performance.head(10)
most_purchasing_vendors['TotalEarned']=most_purchasing_vendors['TotalEarned'].apply(lambda x:round(x/1000000,2))
most_purchasing_vendors['GrossProfit']=most_purchasing_vendors['GrossProfit'].apply(lambda x:round(x/1000000,2))
most_purchasing_vendors['TotalAmountSpent']=most_purchasing_vendors['TotalAmountSpent'].apply(lambda x:round(x/1000000,2))
most_purchasing_vendors
```

Out[27]:

	VendorName	TotalEarned	GrossProfit	TotalAmountSpent	PurchaseContribution%
57	MARTIGNETTI COMPANIES	26.91M	7.36M	19.55M	15.55
25	DIAGEO NORTH AMERICA INC	19.94M	4.04M	15.90M	12.64
46	JIM BEAM BRANDS COMPANY	9.37M	1.51M	7.86M	6.25
68	PERNOD RICARD USA	8.15M	1.34M	6.81M	5.41

	VendorName	TotalEarned	GrossProfit	TotalAmountSpent	PurchaseContribution%
30	E & J GALLO WINERY	9.07M	2.31M	6.76M	5.38
20	CONSTELLATION BRANDS INC	8.28M	1.65M	6.63M	5.27
6	BACARDI USA INC	7.08M	1.33M	5.75M	4.57
79	SAZERAC CO INC	6.32M	1.24M	5.07M	4.03
53	M S WALKER INC	6.34M	1.68M	4.65M	3.70
106	ULTRA BEVERAGE COMPANY LLP	6.36M	1.87M	4.49M	3.57

In [28]:

```
most_purchasing_vendors['PurchaseContribution%'].sum()
```

Out[28]:

```
np.float64(66.37)
```

In [29]:

```
most_purchasing_vendors['CumulativeContribution%']=most_purchasing_vendors['PurchaseCont
most_purchasing_vendors
```

Out[29]:

	VendorName	TotalEarned	GrossProfit	TotalAmountSpent	PurchaseContribution%	Cumulative
57	MARTIGNETTI COMPANIES	26.91M	7.36M	19.55M	15.55	
25	DIAGEO NORTH AMERICA INC	19.94M	4.04M	15.90M	12.64	
46	JIM BEAM BRANDS COMPANY	9.37M	1.51M	7.86M	6.25	
68	PERNOD RICARD USA	8.15M	1.34M	6.81M	5.41	
30	E & J GALLO WINERY	9.07M	2.31M	6.76M	5.38	
20	CONSTELLATION BRANDS INC	8.28M	1.65M	6.63M	5.27	
6	BACARDI USA INC	7.08M	1.33M	5.75M	4.57	
79	SAZERAC CO INC	6.32M	1.24M	5.07M	4.03	
53	M S WALKER INC	6.34M	1.68M	4.65M	3.70	
106	ULTRA BEVERAGE COMPANY LLP	6.36M	1.87M	4.49M	3.57	

In [35]:

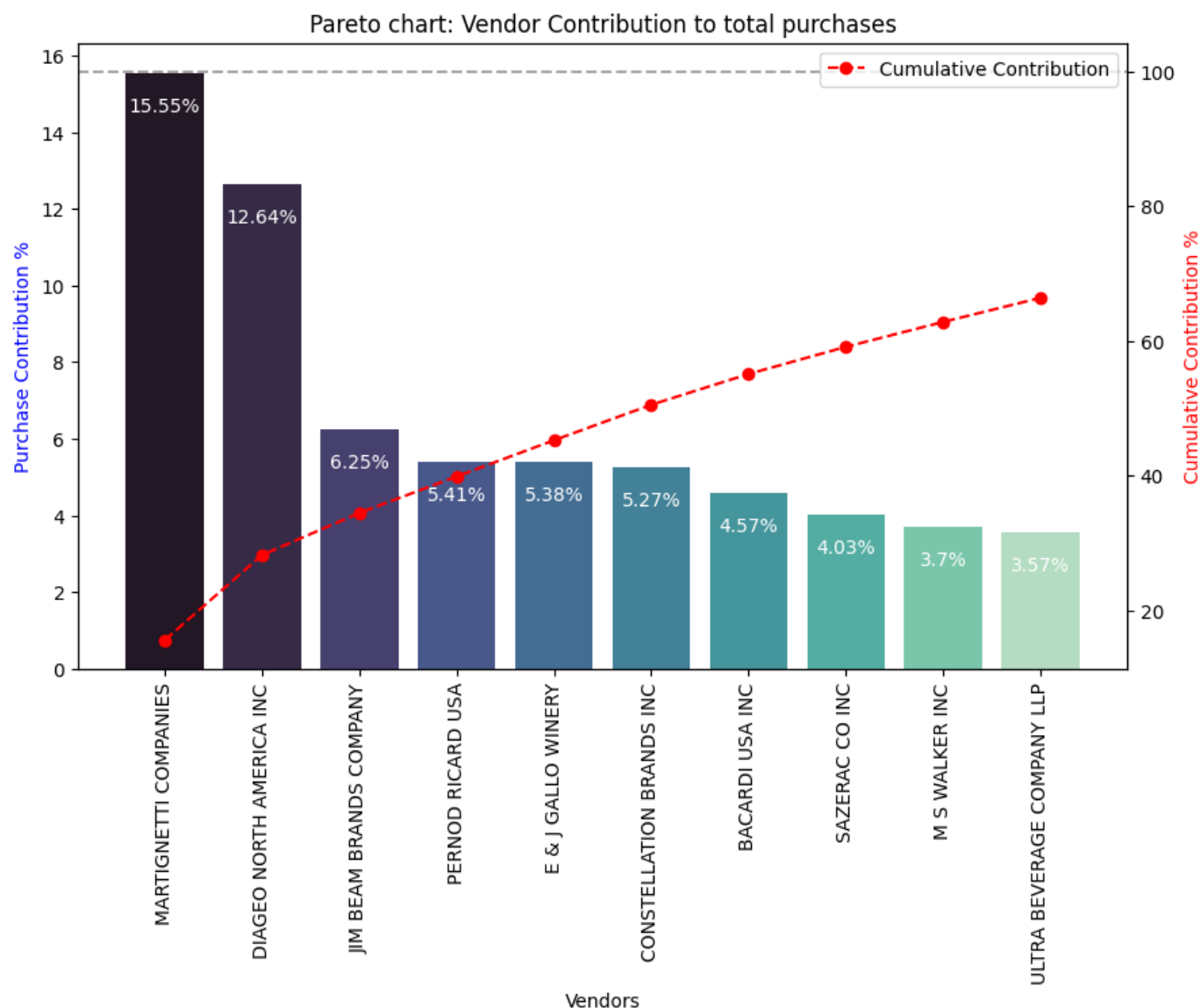
```
fig, ax1=plt.subplots(figsize=(10,6))
# Barplot for purchase contribution:
sns.barplot(x=most_purchasing_vendors["VendorName"],y=most_purchasing_vendors["PurchaseC
```

```

for i,value in enumerate(most_purchasing_vendors["PurchaseContribution%"]):
    ax1.text(i,value-1,str(value)+'%',ha='center',fontsize=10,color='white')

#Line plot for Cumulative Contribution:
ax2=ax1.twinx()
ax2.plot(most_purchasing_vendors["VendorName"],most_purchasing_vendors["CumulativeContri
ax1.set_xticklabels(most_purchasing_vendors["VendorName"],rotation=90)
ax1.set_ylabel("Purchase Contribution %",color='blue')
ax2.set_ylabel("Cumulative Contribution %",color='red')
ax1.set_xlabel("Vendors")
ax1.set_title("Pareto chart: Vendor Contribution to total purchases")
ax2.axhline(y=100,color='grey',linestyle='dashed',alpha=0.7)
ax2.legend(loc='upper right')
plt.show()

```



Pareto Plot Insight:

- Thus, the top 10 most_purchasing_vendors contribute more than 50% of the total purchases.

In [44]:


```

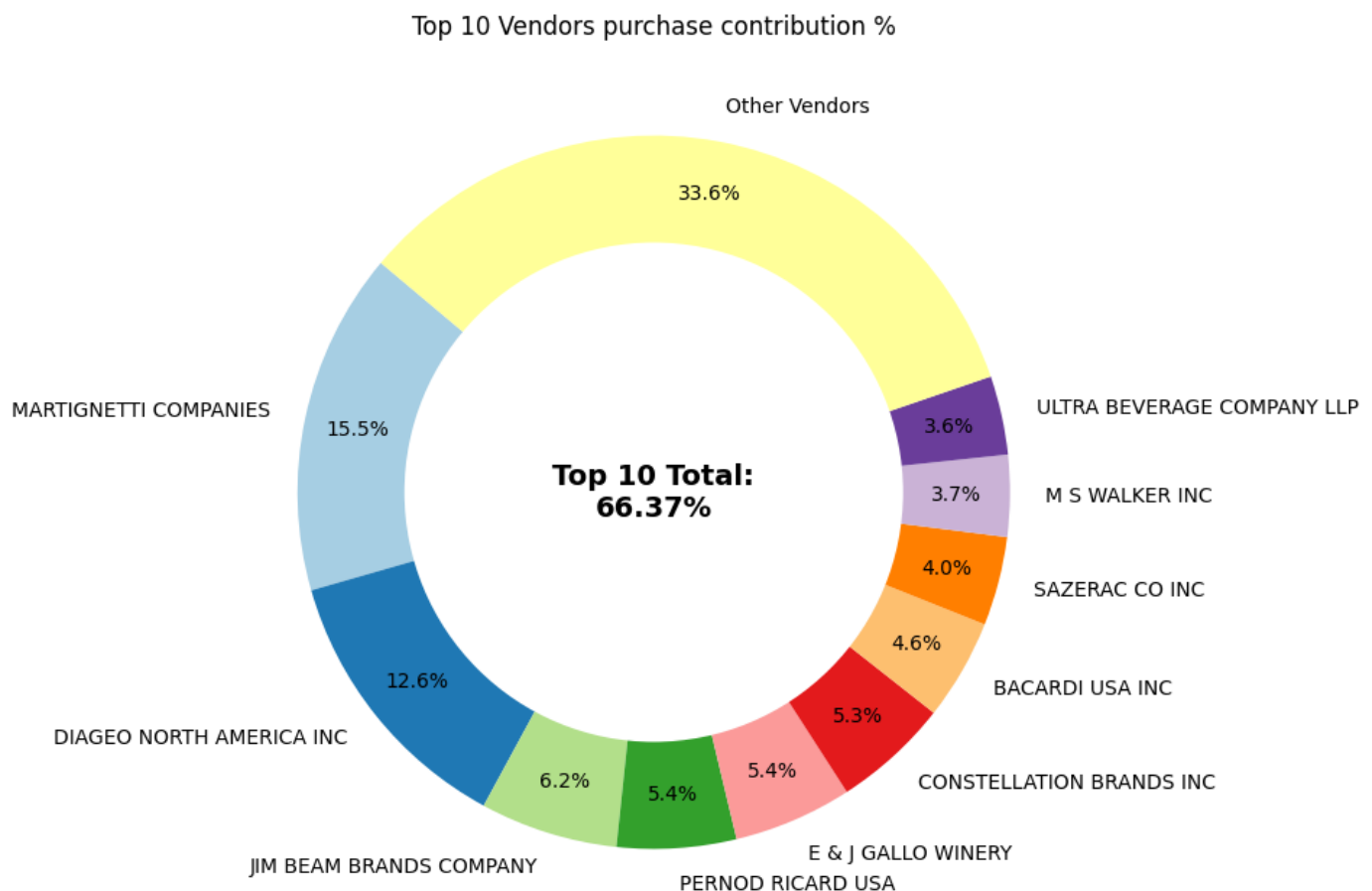
vendors=list(most_purchasing_vendors["VendorName"].values)
purchase_contributions=list(most_purchasing_vendors["PurchaseContribution%"].values)
total_contribution=sum(purchase_contributions)
remaining_contribution=100-total_contribution

# Append Other Vendors as a new element in the list
vendors.append("Other Vendors")
purchase_contributions.append(remaining_contribution)

#Donut Chart, first create a pie chart and then create a white circle to make donut effect
fig,ax=plt.subplots(figsize=(8,8))
wedges, texts, autotexts = ax.pie(purchase_contributions, labels=vendors, autopct='%1.1f%%',
                                  startangle=140, pctdistance=0.85, colors=plt.cm.Paired.colors)

centre_circle=plt.Circle((0,0),0.70,fc='white')
ax.add_artist(centre_circle)
plt.text(0,0,f'Top 10 Total:\n{total_contribution:.2f}%', fontsize=14, fontweight='bold', ha='center')
plt.title('Top 10 Vendors purchase contribution %')
plt.show()

```



Does purchasing in bulk reduce the unit price and what is the optimal purchase volume for cost saving

In [47]:

```
df['UnitPurchasePrice']=df['TotalAmountSpent']/df['TotalQuantityPurchased']
```

In [50]:

```
df['OrderSize']=pd.qcut(df['TotalQuantityPurchased'],q=3,labels=['Small','Medium','Large'])
```

In [53]:

```
df.groupby('OrderSize')['UnitPurchasePrice'].mean()
```

Out[53]:

OrderSize

Small 47.449448

Medium 15.531296

Large 9.846656

Name: UnitPurchasePrice, dtype: float64

- Vendors buying in bulk (Large Order Size) get the lowest unit price(\$9.84 per unit) meaning higher margins if they can manage inventory efficiently.
- The price difference between small and large orders is substantial (~79.2% reduction in unit cost)
- This suggests that bulk pricing strategies successfully encourage vendors to purchase in larger volumes

In []: